

Croco Krave

Memory SECCED Hardware Acceleration for Reliability Improvements

Alessandro Vegni & Giulio Guerrieri
Group 14
ETH Zürich

FS 2026

1 Introduction

For our VLSI 2 project, we focused on the reliability of the *Croc* SoC, aiming to enhance its robustness against ionizing-particle bit-flips — the dominant failure mode in space and high-radiation environments. A software-based SECCED routine would severely limit memory bandwidth: the bit-manipulation overhead of encoding and decoding every read and write transaction is prohibitive. To eliminate this latency, we integrated a byte-level hardware SECCED (Single Error Correction, Double Error Detection) codec directly into the memory interface. The module corrects single-bit errors within the memory-access cycle and flags uncorrectable double-bit errors immediately. The underlying SRAM was widened to 64 bits (to host the parity overhead) and doubled in address space from 512 to 1024 words, while a transparent wrapper maintains the original 32-bit protocol visible to external masters.

For post-silicon validation, two output pins (one per memory bank) expose the double-error flag and three input pins drive a hardware fault-injection engine directly from chip I/O.

2 RTL Design and Implementation

2.1 System-Level Architecture and SECCED Implementation

Figure 1 illustrates the system-level integration of the protected memory subsystem. All memory read and write requests initiated by the processor are routed through the system crossbar directly into the custom memory wrapper. This wrapper encapsulates the SRAM macros along with the combinational logic required for real-time encoding and decoding, ensuring that the hardware-level fault tolerance remains fully abstracted from the rest of the SoC.

2.1.1 Memory Cut Selection and Layout Constraints

The original 512x32 SRAM architecture has been upgraded to a 1024x64 configuration. While the effective 32-bit addressable capacity remains unchanged, the expanded word width is necessary to accommodate the parity overhead required for SECCED protection. Specifically, we implemented the *Hsiao Inverted (13, 8)* algorithm. Applied independently to each byte, this scheme requires 5 parity bits for every 8 bits of data. Consequently, a full 32-bit data word requires 20 parity bits, totaling 52 bits of actual payload per memory line.

To implement this, we selected a standard 64-bit wide SRAM macro from the pre-compiled cuts provided in the IHP PDK, as we had to choose exclusively from the available memory options made available by the foundry. While industrial memory compilers can theoretically generate

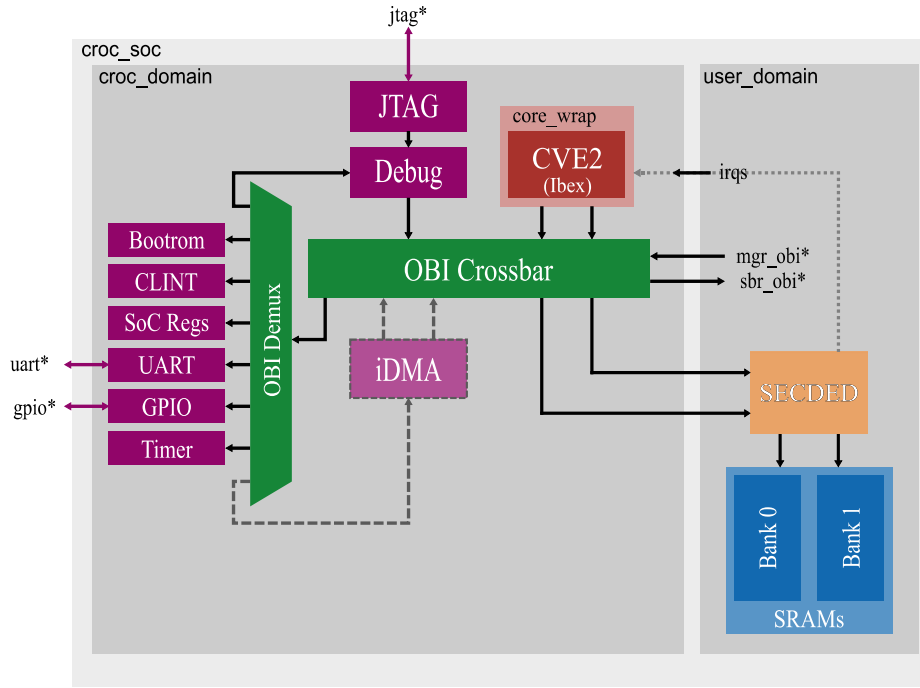


Figure 1: System-level block diagram.

custom SRAMs tailored to an exact width (such as 52 bits), using the standard 64-bit macro was the only practical and compliant choice for our PDK constraints. Because our chip is strictly pad-limited—meaning its physical boundaries are constrained by the required external pins rather than internal logic gates—this over-allocation carries zero penalty. The remaining 12 unutilized bits per word are perfectly and transparently absorbed by the surplus silicon area, with absolutely no impact on the final chip footprint or system functionality.

2.1.2 Algorithmic Scheme: Hsiao Inverted Code Implementation

We chose the Hsiao algorithm over traditional Hamming codes to ensure optimal hardware efficiency. Unlike classic Hamming implementations, which often generate irregular logic trees, the Hsiao scheme features a highly balanced XOR-tree structure. This results in a more balanced critical path and a lower total gate count, significantly reducing propagation latency during the encoding and decoding stages.

The protection mechanism relies on syndrome calculation. During a write operation, the encoder computes the 5-bit parity (syndrome base) for each individual byte and stores it alongside the data. Upon reading, the decoder recalculates these 5 bits using the fetched data and compares them with the stored parity bits to generate a final syndrome. Based on this syndrome, the system reacts as follows:

- **Single-bit error:** If exactly one bit is flipped (either in the data or the parity), the resulting 5-bit syndrome acts as a unique pointer to the exact faulty bit location. The combinational logic uses this pointer to automatically invert the bit back to its correct state before the data reaches the core.
- **Double-bit error:** The mathematical properties of the Hsiao code ensure that any two simultaneous bit flips produce a syndrome that cannot be confused with a single-bit error. When this occurs, the subsystem detects the uncorrectable fault and immediately triggers an interrupt to the controller, preventing the execution of corrupted data.

2.1.3 Protection Granularity: Byte-Level vs. Word-Level Protection

The decision to implement this encoding at the byte level, rather than protecting the entire 32-bit word with a single larger code, was driven by the necessity to maintain native RISC-V byte-addressability. This approach enables individual byte read and write operations to execute directly. If a word-level code had been used instead, writing a single byte would have necessitated a complete Read-Modify-Write cycle—introducing additional finite state machines and relative latency penalties.

While instantiating parallel encoders and decoders for each byte noticeably increases the total logic gate count, it represents a well-justified trade-off for our architecture. Given the aforementioned surplus of available silicon area, paying in terms of combinational logic area to bypass RMW latency is an effective design decision.

2.2 Module Descriptions

The core of our implementation is the `secded_sram_impl` custom wrapper, designed as a drop-in replacement for the original `tc_sram_impl` memory interface. Figure 2 illustrates the simplified view of our design and the integration of its submodules. As illustrated in the block diagram, an additional module, responsible for the silent writeback mechanism, is instantiated in the data path. As later explained, the implementation of the repair logic is highly articulated to preserve the system's native byte addressability. To avoid conflicting write operations, the CTRL block continuously evaluates address collisions and monitors incoming `sram_req`, `sram_we`, and `sram_be` signals to guarantee that background memory repairs execute transparently, ensuring primary CPU memory transactions always maintain strict priority. The design is fully combinational regarding the data processing stages, ensuring it preserves the exact memory cycle latency specified by the original SoC protocol.

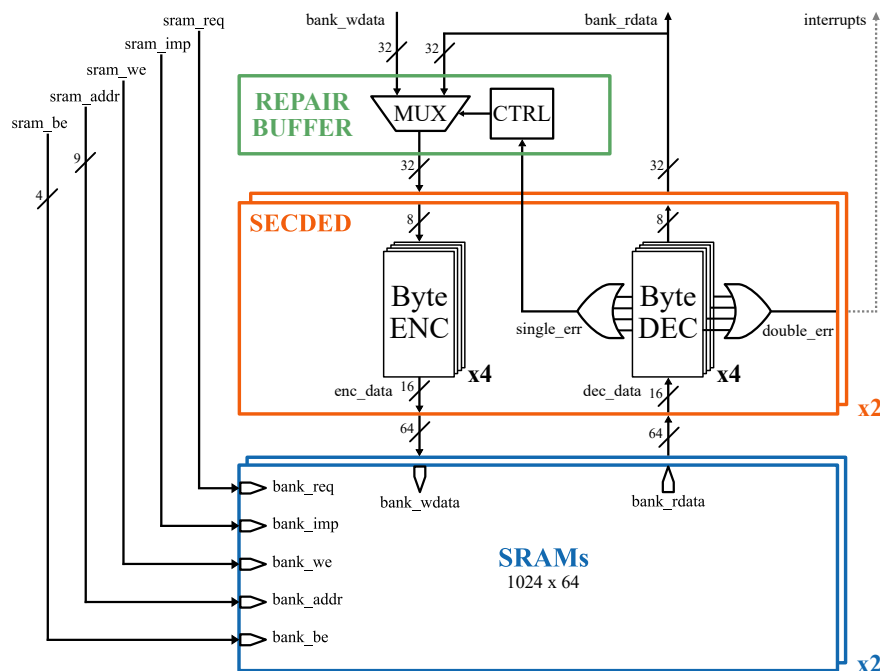


Figure 2: Zoomed-in block diagram of the user domain.

The primary components instantiated within the wrapper to manage fault-tolerance operations are:

- **secded_byte_encode:** This submodule receives an 8-bit data byte and calculates 5 parity bits according to the *Hsiao Inverted (13, 8)* matrix. The resulting 13-bit output constitutes the protected payload.
- **secded_byte_decode:** Situated on the read path, this module receives the 13-bit encoded segment. It recalculates the syndrome bits to verify data integrity. Utilizing an XOR logic network, it generates the original data while asserting either `single_err_o` (for a correctable fault) or `double_err_o` (for a detected, uncorrectable fault).

2.2.1 Data Flow

The wrapper logic operates in parallel across the four bytes that compose the system's 32-bit word, ensuring complete abstraction between the external 32-bit interface and the 64-bit physical memory macro.

Write Path: When the memory controller initiates a write request, the logic decomposes the 32-bit `wdata_i` into four independent bytes. Each byte is processed by an instance of `secded_byte_encode`. To align the 13-bit encoded output with the memory's 16-bit segment width, the module performs a *zero-padding* operation, prepending 3 bits to the payload (`{3'b000, enc_out}`). The resulting 64-bit word is then committed to the SRAM, with the `be_i` (byte enable) signal intentionally mapped to govern these 16-bit chunks.

Read Path: Upon a read operation, the 64-bit line is retrieved and partitioned into four 16-bit segments. The wrapper extracts the 13 relevant bits per segment, discarding the padding, and routes them to parallel instances of `secded_byte_decode`. Detected single-bit errors are corrected in real-time, after which the four bytes are reassembled to reconstruct the final 32-bit `rdata_o`.

Error Handling and Configuration: The `double_err_o` signals from all four decoders are aggregated via an OR-tree. The assertion of this resulting flag immediately raises a hardware interrupt that is routed directly to the processor core. This mechanism guarantees that the CPU halts execution before consuming the permanently corrupted data, thereby preventing unpredictable behavior or avoidable system failures.

Single Error Repair Buffer: When a bit flip occurs in the SRAM during a CPU read operation, the SECDED decoder detects and corrects the single-bit error on the fly, providing the corrected data to the processor. However, the data residing within the physical memory remains corrupted. If a second bit flips within the same encoded byte before being repaired, it results in an uncorrectable double-bit error. To maintain fault tolerance over consecutive cycles, the system must perform a writeback operation in the SRAM of correct data.

To achieve this without stalling the processor, the `secded_repair_buffer` latches the corrected word from the decoder and holds it until the next idle cycle, executing the writeback as soon as there are no active write requests. If the faulty memory address is overwritten in the meantime, the buffer detects the collision and invalidates the pending writeback request. Furthermore, because the memory is byte-addressable, the writeback mechanism selectively updates only the corrupted byte(s).

2.2.2 External Signal Routing and Testability

The double-error flags and fault-injection controls introduced in §1 and §2.1 are mapped to specific GPIO pads on the physical right side of the chip, as shown in Figure 3:

- **Double Error Output Flags:** When a fatal double-bit error is detected, Bank 0 and Bank 1 drive the output flags directly to `gpio_o[20]` and `gpio_o[15]`, respectively.
- **Fault Injection Input Controls:** To perform controlled error injection, `gpio_in[19]` and `gpio_in[16]` target errors into Bank 0 and Bank 1. A third pin, `gpio_in[18]`, selects the fault type (0 for single-bit, 1 for double-bit error).

To prevent metastability and glitching, these external inputs traverse a synchronization block in the `croc_domain` before entering the `user_domain`. Due to this, the control inputs reach the destination fault-injection hardware with a maximum delay of one clock cycle relative to the external instantiation, ensuring safe runtime evaluation without compromising system integrity.

2.2.3 Timing Optimization via Inter-Domain OBI Cut

During physical synthesis and initial timing closure analysis, the inclusion of the combinational SECDDED blocks significantly increased the propagation delay across the data read and write paths stretching from the processor core to the memory subsystem, resulting in a negative slack on the setup time.

To shorten the longest combinational path, we integrated an `obi_cut` module as an isolation and pipeline stage within the crossbar interface, positioned directly between the `croc_domain` and the `user_domain`.

Equipped with internal spill registers, the `obi_cut` breaks the long, cross-domain combinational paths into two distinct pipeline segments, as illustrated in Figure 4. Although this modification introduces a controlled single-cycle latency overhead for cross-domain bus transactions, it was essential to close timing without an excessive clock frequency reduction. By optimizing this boundary, we minimized the performance penalty and achieved stable, robust timing closure during back-end physical implementation.

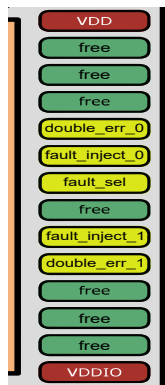


Figure 3: Pin Mapping.

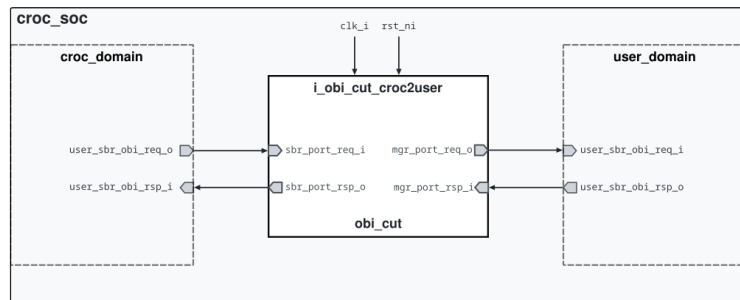


Figure 4: Block diagram of `obi_cut` implementation between the core and user domains.

2.3 Configurable SECDDED Feature Toggle

For architectural flexibility and project reversibility, a feature toggle is implemented via the global `SECDDEDByypass` parameter inside `user_pkg`. This parameter structurally alters the synthesized topology as follows:

- **Active Bypass (1'b1):** Reverts the design entirely by disabling SECDDED logic and instantiating the original, standard 32-bit wide SRAM macro, leaving the original Croc memory interface.
- **Normal Operation (1'b0):** Activates our custom improvements, instantiating the expanded 64-bit physical memory along with the parallel encoding and decoding hardware.

This mechanism allows the system to switch seamlessly between the original chip baseline and our modified version, facilitating performance comparison and design reversibility.

2.4 Functional Verification

To verify the functional correctness and robustness of the enhanced memory subsystem, we adopted an incremental, bottom-up verification strategy. Testing the design starting from the low-level primitive modules and progressively expanding outward to the system level allowed us to isolate and resolve logical issues systematically.

2.4.1 Unit-Level Codec Verification

We initiated verification with the foundational core of our design: the standalone byte encoder and decoder submodules. Because the input space of a single data byte is bounded to 2^8 permutations, we implemented an exhaustive functional verification approach rather than relying on pseudo-random stimulus coverage. A deterministic testbench (`tb_secdded_byte`) subjects the `secdded_byte_encode` and `secdded_byte_decode` blocks to every mathematically possible combination of data inputs and error vectors by programmatically applying an `error_mask` across all 13 positions of the encoded word.

The total combinatorial state space evaluated by the testbench is calculated as follows:

$$N_{\text{total}} = 2^8 \times \left(1_{\text{clean}} + \binom{13}{1}_{\text{single}} + \binom{13}{2}_{\text{double}} \right) \quad (1)$$

$$N_{\text{total}} = 256 \times (1 + 13 + 78) = 23,552 \text{ unique test vectors} \quad (2)$$

Due to the purely combinational nature of the codec submodules, simulating all 23,552 test conditions incurs negligible computational overhead, executing fully in under a second. The logic validation is structurally partitioned into three deterministic scenarios:

- **Roundtrip Integrity:** Validates all 256 clean data words to confirm that uncorrupted data passes through encoding and decoding symmetrically without generating false error flags.
- **Single-Bit Error Correction:** Evaluates 3,328 vectors by systematically flipping every single bit position across all data inputs. The testbench confirms that the decoder asserts `single_err_o`, identifies the corrupted bit index, and achieves a 100% bit-accurate reconstruction of the input.
- **Double-Bit Error Detection:** Evaluates 19,968 vectors by injecting all 78 possible dual-bit flip combinations per word. The testbench verifies that the decoder reliably detects uncorrectable faults and asserts the `double_err_o` flag.

Ultimately, the `tb_secdded_byte` suite achieved a 100% pass rate with zero mismatches across all 23,552 vectors. Every error flag strictly matched mathematical expectations, confirming the absolute functional correctness of the codec logic prior to its integration within the memory wrapper.

2.4.2 System-Level Testing and Fault Injection Simulation

To verify system-level behavior, faults were actively injected into the memory macro. As detailed in §2.2.2, the verification environment drives runtime error generation by mapping synchronized GPIO inputs directly to the internal SRAM control interface.

During a write operation, the target data word undergoes a bitwise XOR operation with one of two hardcoded error masks, depending on the selected fault type:

```
localparam logic [63:0] SecMaskSingle = 64'h0008_0004_0002_0001;
localparam logic [63:0] SecMaskDouble = 64'h00c0_0030_000c_0003;
```

These masks inject exactly one or two bit-flips into different positions of each encoded byte segment.

We validated this hardware infrastructure by executing the `bitflip_verify.c` firmware on the processor core within the `tb_fault_injection.sv` testbench environment. In all test scenarios, the single-error status flag and the double-error hardware interrupt were correctly asserted, proving that the full system responds accurately to memory corruption.

For deterministic simulation, we instantiated a modified top-level module (`croc_soc_sim`) in the testbench in place of the standard `croc_soc`. This simulation-only variant routes the injection signals directly into the `user_domain`, bypassing the hardware synchronization stage. This bypass eliminates the one-cycle synchronization latency, guaranteeing that faults align precisely with the processor’s memory write instruction boundaries during verification.

2.4.3 Repair Buffer Verification

To verify the functional behavior of the automated memory repair buffer during execution, we evaluated the `sec_repair.c` firmware inside a specialized testbench environment (`tb_sec_repair.sv`). The testing sequence follows three main steps:

1. A data pattern is written to a target memory address while the fault injection logic is enabled, forcing a single-bit error into that line.
2. The processor core performs an initial read from this address. The memory wrapper captures the corrupted entry, asserts the single-error flag, corrects the data in real-time for the processor, and schedules a background write-back operation to repair the SRAM entry.
3. The processor core waits for a few clock cycles to allow the repair buffer to complete the background write-back, and then executes a second read from the exact same address.

By tracking the internal error signals, we confirmed that the single-error flag was asserted only during the first read transaction and remained low during the second access. This confirms that the wrapper corrected the payload and used the idle cycles between transactions to write the corrected data back into the SRAM macro, verifying correct repair-buffer operation.

3 Performance Evaluation (Front-End)

To quantify the performance benefits of our hardware acceleration, we conducted a comparative analysis between the hardware-integrated SECDED subsystem and an equivalent software-based implementation executed on the processor core. The objective was to demonstrate the substantial overhead that software-based fault tolerance imposes on memory access compared to the highly optimized latency of our custom hardware solution.

3.1 Evaluation Setup and Baseline

The evaluation was driven by custom C benchmarks running on the *Croco Krave* processor. Our methodology relied on counting the clock cycles required to execute the algorithm via software versus our hardware implementation across three distinct scenarios: zero errors, a single correctable error, and a double uncorrectable error per word. The total cycles were averaged over a large number of samples (`NUM_SAMPLES`) to ensure measurement stability.

To establish an authentic comparison, we executed the benchmarks across two completely distinct simulation configurations:

- **Software Baseline Configuration:** We activated the global `SECDEDBypass` configuration flag in `user_pkg.sv`. This disabled the hardware accelerator, reverting the chip to its original state with the native 32-bit SRAM implementation and bypassing the `obi_cut` pipeline

stage. In this pure-software setup, storing a single protected word (which mathematically expands to 64 bits) forced the core to manually allocate and manage two distinct 32-bit memory addresses.

- **Hardware Accelerated Configuration:** To measure the true cycle count of our enhanced design, we disabled the `SECDEDBypass` flag. This automatically instantiated the full target architecture, including the custom hardware SECDED wrapper, the underlying 64-bit memory macro, and the active `obi_cut` module.

To track execution cycles accurately, the benchmarks signal the start and stop of the operations by writing to specific Memory-Mapped I/O (MMIO) addresses in the `USER_DESIGN` peripheral space. Because of this signaling mechanism, a standard read/write operation in the software baseline (without `obi_cut`) inherently requires 7 cycles of transaction overhead. In the hardware configuration, the `obi_cut` pipeline stage (§2.2) increases this baseline to 15 cycles.

Since integrating the SECDED datapath lengthened the critical timing arc of the system, to prevent setup violations and secure timing closure, the operating clock period had to be proportionally relaxed, reducing the maximum clock frequency from 100 MHz to approximately 73 MHz. Despite these architectural constraints, the hardware implementation still outperforms the software baseline by fully counteracting these physical trade-offs.

3.2 Quantitative Results and Speedup

Table 1 summarizes the performance comparison between the software baseline (operating at 100 MHz) and the hardware-accelerated system (operating at 73 MHz), divided into raw cycle counts and absolute execution time.

(a) Cycle Count Analysis

Scenario	SW Cycles	HW Cycles	Cycle Speedup
Encode / Store	267	15	17.8×
Decode / Load (0 Errors)	518	15	34.5×
Decode / Load (1 Error)	657	15	43.8×
Decode / Load (2 Errors)	660	15	44.0×

(b) Absolute Execution Time Analysis

Scenario	SW Time (@100MHz)	HW Time (@73MHz)	Time Speedup
Encode / Store	2.67 μ s	0.21 μ s	13.0×
Decode / Load (0 Errors)	5.18 μ s	0.21 μ s	25.2×
Decode / Load (1 Error)	6.57 μ s	0.21 μ s	32.0×
Decode / Load (2 Errors)	6.60 μ s	0.21 μ s	32.1×

Table 1: Performance Comparison: Software Baseline vs. Hardware Accelerated SECDED.

The results demonstrate a massive reduction in latency across all operations. Notably, in the software baseline, there is a significant discrepancy between the encoding execution time (267 cycles) and the decoding execution time (518 to 660 cycles). This is due to the inherent algorithmic complexity of the read path: while encoding merely calculates and appends parity bits, decoding forces the CPU to re-calculate the syndrome, compute its logical weight, execute conditional branches to isolate the specific error position, and finally apply a bitwise correction mask.

Conversely, the hardware accelerator executes the entire Hsiao algorithm through a parallelized combinational logic network. Even with the necessary frequency reduction to 73 MHz to meet

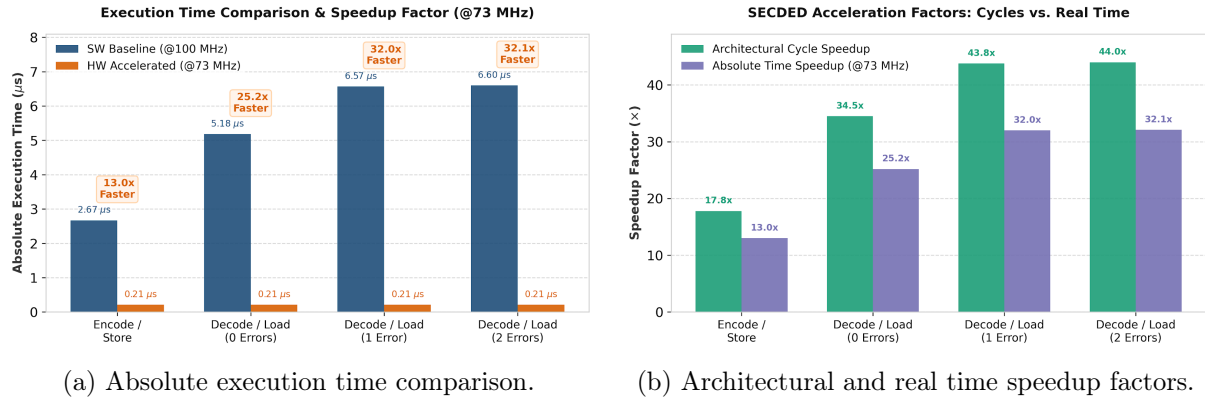


Figure 5: Performance comparison charts detailing the software baseline versus hardware-accelerated execution profiles.

the physical timing constraints, the hardware achieves an execution time of approximately 205.5 nanoseconds per memory transaction regardless of the error state, as visually confirmed by the constant hardware baseline in Figure 5a.

Ultimately, replacing the software routine with dedicated hardware logic yields an absolute time speedup ranging from **13.0x (for clean writes) up to 32.1x (for faulty reads)**, alongside a raw architectural cycle reduction of up to **44.0x**, as illustrated in Figure 5b. This proves that hardware offloading is strictly necessary to maintain a viable memory bandwidth and overall system throughput.

3.3 Functional and Security Limits of Software SECCED

Beyond pure latency, a software-only SECCED approach carries a fundamental, self-referential vulnerability: it cannot protect the memory regions hosting the processor’s core programs, bootloaders, or the error-correction routine itself. If a bit-flip strikes that region, the handler is permanently compromised before it can execute, leading to an unrecoverable system crash.

Consequently, software creates a dangerous "blind spot" in the very subsystem meant to provide robustness. Only a hardware interface can transparently safeguard the entire memory map from the first boot cycle, eliminating this dependency and ensuring comprehensive system integrity.

4 Back-End Implementation

4.1 Synthesis - *Yosys*

During netlist generation using the open-source *Yosys* tool, particular attention was paid to the flattening strategy. To preserve a structured and detailed view of the architecture, the design hierarchy was maintained wherever possible. This approach allowed for a clear distinction between the various instantiated blocks, specifically enabling accurate area accounting and tracking of the `user_domain` relative to the rest of the system. Although restricting the flattening scope limits the tool’s capacity for cross-boundary logic optimization and restructuring, maintaining modularity was prioritized for design traceability.

To guarantee a robust timing margin for the subsequent backend implementation phases, the synthesis clock period constraint was intentionally over-constrained, enforcing a target that was approximately 1 ns tighter than the actual operational target of 13.7 ns (73 MHz). While over-constraining may enhance chip performance, our initial synthesis results revealed that an overly aggressive target severely degrades design efficiency. Under tight constraints, *Yosys* excessively duplicates logic and inserts massive buffer trees, driving up area and power overheads.

4.2 Placement & Routing - *Openroad*

4.2.1 Floorplan

Since the newly instantiated memory macros feature a significantly larger physical footprint due to increased word width and depth, their placement strongly constrains the remaining area available for routing. This required accurate floorplan tuning to optimize performance and prevent severe congestion. Following extensive layout iterations, the optimal configuration involved placing the macros at opposite vertical extremes of the core area, slightly shifted to the left towards the clock pin, as shown in Figure 6a. Furthermore, the macros were rotated so that their signal inputs faced inward toward the core logic, thereby minimizing total wire length and routing overhead.

Additional modifications were made to the pad ring to comply with the new area specifications. Although Figure 3 illustrates the adjustments made to the pin layout, the `gpio_x` naming convention was preserved for simplicity, and the routing remains entirely internal to the `croc_soc` module. Lastly, adjustments were made to the power grid architecture. These changes and their respective outcomes will be discussed in detail in the subsequent IR drop analysis section.

4.2.2 Placement

Placement optimization was performed via a parametric sweep of the target utilization density from 55% to 70%. A density of 65% yielded the optimal performance in terms of Design Rule Violations (DRVs), producing the absolute minimum fan-out across all evaluated configurations. A detailed discussion of these DRV metrics is provided in the Placement & Routing Conclusions section. As illustrated in Figure 6b, this specific balance prevented local cell overcrowding while maintaining short interconnect lengths. Additionally, an automated mitigation strategy was implemented using the `set_placement_padding` command to achieve a clean DRC report. Because the insertion of cell padding directly impacts local utilization, this technique slightly modified the final effective density within the core area.

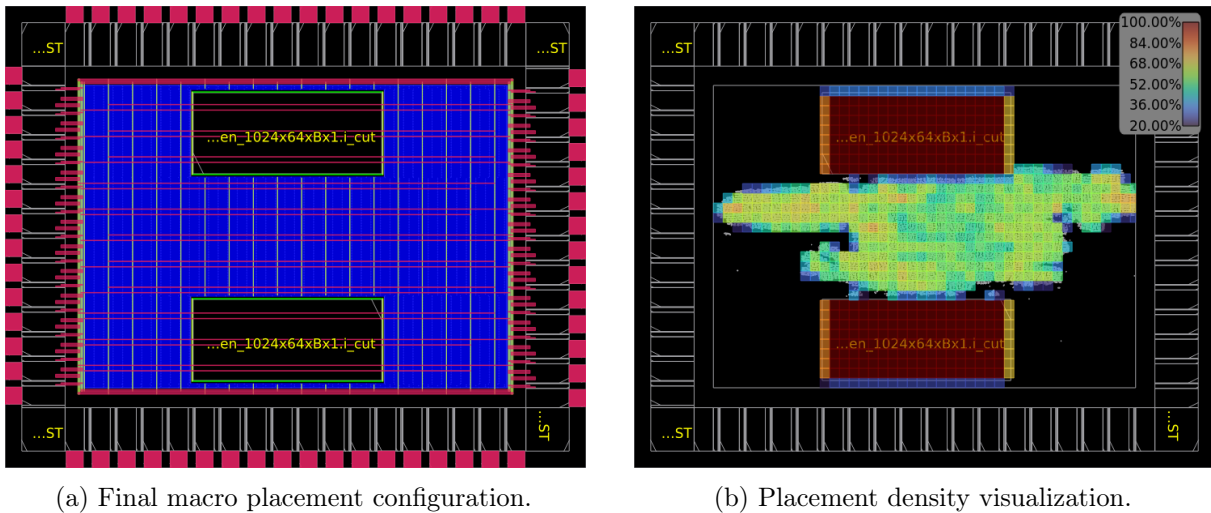


Figure 6: Global placement and density analysis.

4.2.3 Clock Tree Synthesis & Routing

The Clock Tree Synthesis (CTS) phase required an iterative approach to balance strict timing constraints with physical design limitations. While the primary objective of tuning the CTS parameters was to achieve timing closure and minimize skew, the resulting architecture also mitigated dynamic IR drop. By enforcing balanced clock levels, the synthesis engine distributed the clock buffers more evenly across the core. This prevented buffer congestion near macro

boundaries, effectively reducing localized voltage drops. Furthermore, combining a conservative maximum load capacitance margin with strict sink clustering limits forced the tool to utilize higher-drive buffers. This effectively constrained the logical fanout and improved the slew rate, thereby sharpening the signal transition edges. The final clock tree structure is shown in Figure 7a. The direct power overhead resulting from these adjustments is analyzed and discussed in the subsequent power analysis section.

During the physical routing stage, it was observed that automated antenna repair mechanisms—such as diode insertion or routing layer jumps—frequently introduced new design rule violations. A multi-pass detailed routing strategy involving dedicated antenna repair steps was investigated; however, as it yielded suboptimal results, it was ultimately not applied. The standard detailed routing flow was therefore retained, resulting in the final layout depicted in Figure 7b.

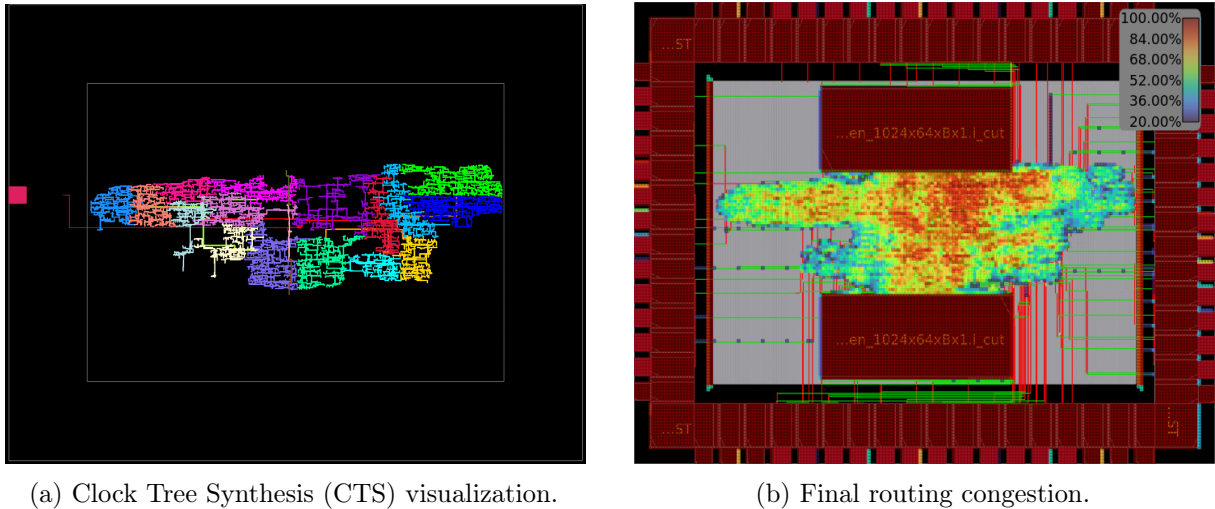


Figure 7: CTS and routing analysis.

4.2.4 IR Drop

The design utilizes the 130 nm technology node, which inherently provides structural advantages regarding power integrity and voltage distribution. Following the modifications implemented in the previous steps, the baseline static IR drop was measured at approximately 0.88%. In industrial practices, a threshold of 2% is typically required for robust Power Delivery Networks, whereas values exceeding 5% are generally considered critical. Despite operating within these comfortable margins, further minimizing the IR drop was pursued as an additional optimization exercise. To this end, the power grid was initially redesigned with a tight pitch and wider wires on both Metal 4 and Top Metal 1. This highly conservative configuration successfully drove the static IR drop down to an exceptionally low 0.64% for the VDD net.

However, because reducing the IR drop to such an extreme extent was not strictly necessary, the constraints were relaxed to better balance power integrity and routing resources. By increasing the grid spacing and reducing the wire widths of both Metal 4 and Top Metal 1, as shown in Figure 6a, the static IR drop for the VDD net was brought to 1.19%, which remains below the acceptable threshold of 2%. This strategic adjustment freed up significant routing tracks, granting the placement and routing engines more physical resources to optimize the overall design and alleviate local routing congestion.

The final heatmap alongside the respective IR drop reports for both VDD and VSS nets at the typical corner (tt) are shown in Figure 8 and the adjacent tables.

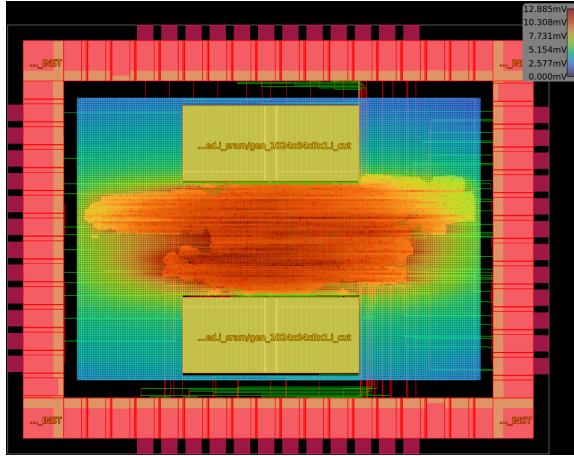


Figure 8: IR-drop distribution for net VDD.

IR Report - VDD

Worstcase voltage	1.19e+00 V
Average voltage	1.19e+00 V
Average IR drop	9.13e-03 V
Worstcase IR drop	1.42e-02 V
Percentage drop	1.19 %

IR Report - VSS

Worstcase voltage	1.29e-02 V
Average voltage	8.97e-03 V
Average IR drop	8.97e-03 V
Worstcase IR drop	1.29e-02 V
Percentage drop	1.07 %

4.2.5 Placement & Routing Conclusions

Throughout the physical design loop, timing closure and design rule compliance were rigorously monitored using both post-routing and post-parasitic extraction (RC extraction) reports, taking into account the distinct validity and confidence levels of each stage.

Hold Time As an absolute priority, we ensured that zero hold violations remained across both the post-routing and post-extraction reports. Eliminating hold slack violations is critical for sign-off, as it guarantees that the fabricated silicon will be free of catastrophic race conditions and remain fundamentally operational.

Setup Time Regarding setup timing, an optimal layout "sweet spot" was achieved post-routing, yielding zero setup violations at a target clock frequency of 73 MHz. This robustness is maintained not only under nominal conditions (**tt**) but also across extreme PVT variations, specifically the Fast-Fast (**FF**) and Slow-Slow (**SS**) verification corners, safeguarding the operation against worst-case environmental and process shifts. However, the subsequent post-parasitic extraction report revealed setup violations at the same frequency, which are largely attributed to pessimistic RC estimation rather than definitive physical failure. Ultimately, operational robustness of the chip is fully preserved; if necessary, the clock frequency can simply be scaled down to eliminate any setup timing violations.

Maximum Frequency Post-routing analysis reported a positive setup slack of +0.0997 ns, yielding a theoretical minimum clock period of:

$$T_{min_routed} = 13.7 \text{ ns} - 0.0997 \text{ ns} = 13.6003 \text{ ns}$$

This translates to an estimated maximum operating frequency of approximately 73.5 MHz.

However, following full parasitic extraction (SPEF), interconnect RC delays caused the setup slack to drop to -4.6841 ns. To achieve timing closure without further physical optimization, the clock period must be relaxed to accommodate this overhead:

$$T_{min_pex} = 13.7 \text{ ns} + 4.6841 \text{ ns} = 18.3841 \text{ ns}$$

This severe degradation restricts the frequency to approximately 54.4 MHz.

Consequently, while the post-routing estimation indicates a performance ceiling of 73.5 MHz, the maximum compliant frequency under the full post-extraction model is strictly constrained to the 54.4 MHz. This degradation closely relates to the baseline Croc implementation, where

the maximum frequency similarly dropped from 100 MHz post-routing to 72.5 MHz after full parasitic extraction.

Design Rule Violations (DRVs) Design Rule Violations (DRVs) were recursively analyzed using the `report_check_types -X -violators` command, which enabled the direct identification of the specific cells responsible for each violation. Consequently, through the previously discussed modifications during placement and routing, post-routing fan-out violations were successfully minimized, reducing the count from 122 to just 5. This represents a substantial improvement in fan-out structural integrity compared with our first iteration and the reference flow itself.

Extensive effort was dedicated to optimizing signal slew. Because the custom memory macros are double the size of the baseline in both word width and depth, driving their input pins became significantly more challenging due to the increased capacitive load. While we successfully reduced the overall number of slew violations on the macros' bitmask signals compared to the reference flow, the severity of the worst-case violations increased. These critical violations persisted despite multiple optimization cycles aimed at forcing high-drive buffers and mitigating local routing congestion around the macro pins.

4.3 DRC - *KLayout*

The final step in the physical implementation flow was the Design Rule Checking (DRC) verification, which ultimately achieved a completely clean report. As previously noted in the placement section, the utilization of automated placement padding successfully mitigated local routing congestion, leaving only a single localized M1.b layer violation. This remaining error was subsequently resolved through targeted manual geometry modifications.

4.4 LVS - *Calibre*

Similarly, the Layout Versus Schematic (LVS) check was successfully passed, confirming perfect structural consistency between the final post-routing gate-level netlist and the physical layout geometry. Following this successful LVS sign-off, the finalized physical design was exported in the standard GDSII format as `croc.gds`.

5 Physical Metrics

To ensure a consistent and realistic evaluation, all physical implementation analyses presented in this section were conducted under the nominal PVT (Process, Voltage, Temperature) corner. Specifically, the design was characterized using the typical-typical (`tt`) process corner at a supply voltage of 1.20 V and a temperature of 25 °C.

5.1 Area Analysis

As presented in Fig. 9a, the total layout footprint expands significantly from 314,200 μm^2 to 861,500 μm^2 . The breakdown in Fig. 9b isolates the root cause of this physical growth: while standard cell logic undergoes a standard proportional increase (reaching 332,287 μm^2), the embedded memory subsystem experiences a massive expansion, surging from 125,400 μm^2 to 529,213 μm^2 . These newly instantiated, high-capacity SRAM macros feature a drastically enlarged physical footprint due to expanded word width and depth. This shifts the chip's area composition: while the memory subsystem originally accounted for roughly 40% of the baseline area, it now dominates our implementation, occupying over 60% of the final footprint.

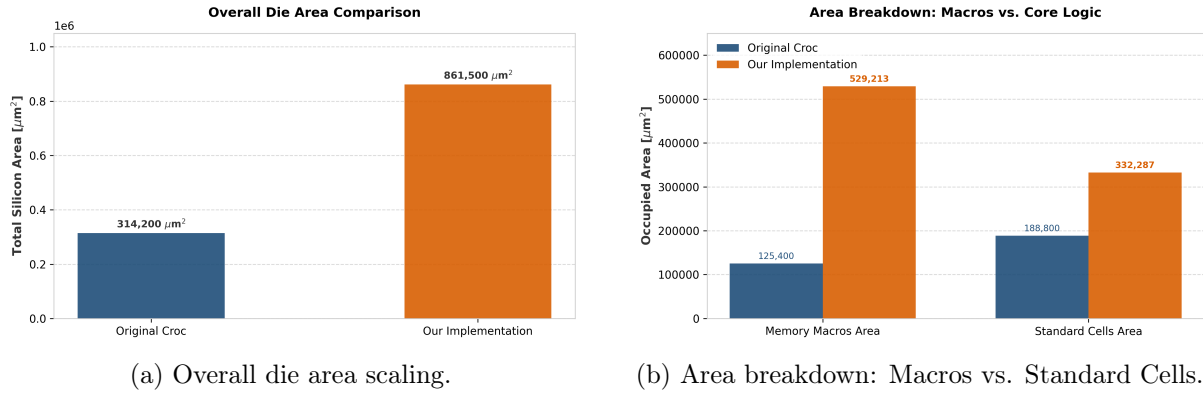


Figure 9: Area comparison between Croco Krave and the original Croc implementation.

Figure 10 provides a detailed perspective on the physical and hierarchical area impact of our implementation. In Fig. 10a, the additional hardware introduced by the SECDED logic and repair buffers is visualized directly on the final silicon layout. To complement this, the hierarchical breakdown in Fig. 10b quantifies the exact area distribution within the subsystem. While the two expanded SRAM banks dominate the layout footprint, their reported area also includes the SECDED codec logic, as they were placed within the same hierarchical level to maximize resource optimization during synthesis. The standalone fault-tolerance infrastructure introduces a remarkably low area overhead, with the dual repair buffers accounting for a mere 0.5% each (1% total) of the final design area.

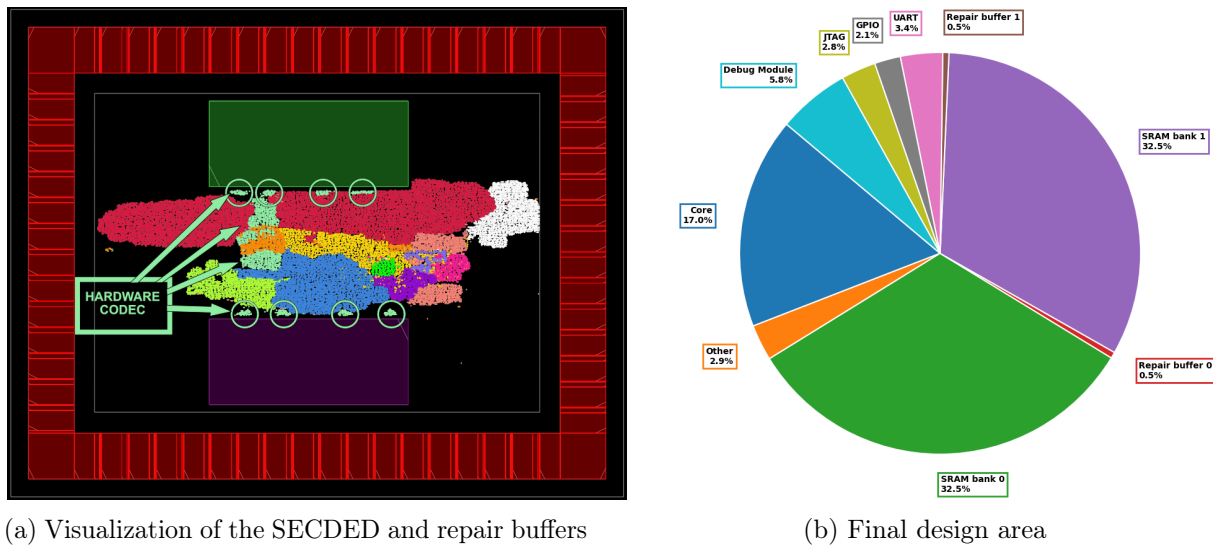


Figure 10: Hierarchy and area visualization

5.2 Power Analysis

To evaluate the energetic efficiency of the design, power analysis was performed using two distinct methodologies: *Statistical Power Analysis* and *Stimuli-Based Power Analysis*. This dual-approach provides both a generalized architectural baseline and a highly accurate estimation under realistic operational workloads.

Statistical power analysis

Statistical (vectorless) power analysis estimates the power consumption of the netlist without requiring time-consuming functional simulations. Instead, it relies on a probabilistic approach

where a static probability and a switching activity factor are propagated through the logic gates. For this analysis, a baseline toggle rate was established by enforcing a uniform 10% switching activity across all internal nodes using the `-activity 0.1` constraint in the power analysis tool. While computationally efficient, this method provides a generalized power profile rather than capturing application-specific behavior. The resulting power estimation and group-specific breakdown obtained through this approach are summarized in Table 2.

Group	Internal [mW]	Switching [mW]	Leakage [mW]	Total [mW]	%
Sequential	5.27	0.158	1.30e-3	5.43	27.2 %
Combinational	0.200	0.505	1.59e-3	0.706	3.5 %
Clock	6.91	3.25	0.79e-3	10.20	51.0 %
Macro	63.6e-03	0.00	1.58e-3	65.2e-03	0.3 %
Pad	0.282	3.30	0.11e-3	3.58	18.0 %
Total (%)	12.70 (63.8 %)	7.21 (36.1 %)	5.37e-3 (0.03 %)	19.90 (100 %)	

Table 2: Statistical power analysis

As shown in Figure 11a, the total power consumption compared to the original architecture increases by 8.15% (rising from 18.40 mW to 19.90 mW). The breakdown in Figure 11b reveals a reduction in sequential and combinational logic power, achieved through effective cross-boundary optimization. However, this gain is entirely offset by a significant surge in the clock network power, which rises from 6.89 mW to 10.20 mW, accounting for 51.0% of the total power budget. This penalty represents the overhead of aggressive Clock Tree Synthesis (CTS) and intensive buffer insertion required to resolve hold violations around the relocated, larger memory macros. Conversely, pad and macro power contributions remain essentially unchanged.

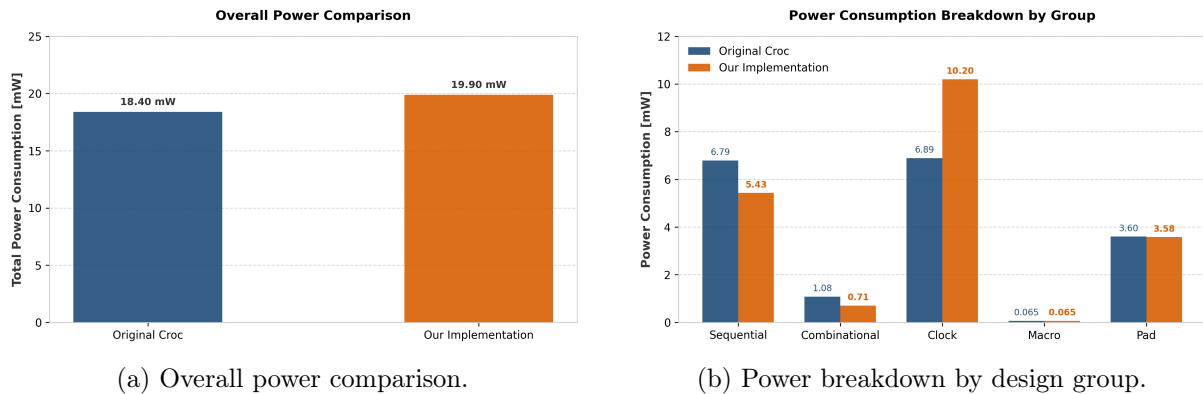


Figure 11: Power comparison between Croco Krave and the original Croc implementation.

Stimuli-Based Power Analysis

To achieve a highly accurate and realistic power characterization, a stimuli-based (vector-driven) power analysis was conducted. This methodology replaces statistical assumptions with deterministic switching data extracted directly from a simulation of a representative workload. Specifically, the test program `sedced_print.hex` was executed to replicate a characteristic operating scenario, with the resulting power breakdown summarized in Table 3.

Group	Internal [mW]	Switching [mW]	Leakage [mW]	Total [mW]	%
Sequential	5.24	0.124	1.30e-3	5.36	27.4 %
Combinational	0.148	0.289	1.59e-3	0.439	2.2 %
Clock	6.91	3.25	0.79e-3	10.20	51.8 %
Macro	63.6e-03	0.00	1.58e-3	65.2e-03	0.3 %
Pad	0.281	3.30	0.11e-3	3.58	18.2 %
Total (%)	12.60 (64.5 %)	6.95 (35.5 %)	5.37e-3 (0.03 %)	19.60 (100 %)	

Table 3: Stimuli-based power analysis

6 Conclusions

Summary of Achieved Improvements

The integration of a hardware-accelerated SECDED codec successfully eliminated the latency of software-based routines, achieving up to a **44.0x** cycle reduction and a **32.1x** absolute time speedup. By implementing the Hsiao algorithm in hardware, the entire memory map is transparently protected from the first boot cycle. Furthermore, the final design successfully achieved clean DRC and LVS verification with zero hold violations.

Area and Power Constraints

Operating within strict physical limits required careful management of area and power trade-offs. The total silicon footprint expanded from 314,200 μm^2 to 861,500 μm^2 , driven almost entirely by the larger 64-bit SRAM macros. Similarly, total power consumption saw a marginal 8.15% increase (up to 19.90 mW), predominantly localized in the clock network. These overheads represent a highly justifiable trade-off for the massive gains in system reliability and throughput.

Lessons Learned

A major takeaway from this project was the hands-on experience with an open-source EDA toolchain. Utilizing these platforms provided a highly accessible environment to navigate complex physical design flows, proving especially valuable for those without prior experience with proprietary industrial software. Furthermore, executing the entire design cycle—from the initial architectural concept and RTL implementation, to the software baseline comparison, and finally the back-end execution requiring an iterative balance of placement, routing, and strict timing and DRV constraints—offered practical insights into the real-world challenges of chip design.

Thanks for your attention